



Architettura degli elaboratori

Lezione 11 - esercitazione

Dott. Gianmarco Cuciniello

Università degli Studi di Milano



Contenuto

- Riepilogo istruzioni
- CPUlator
- Esempi
- MIPS esercizi pratici



Riepilogo istruzioni (1)

Operandi MIPS

Nome	Esempio	Commenti
32 registri	$\$s0-\$s7$, $\$t0-\$t9$, $\$zero$, $\$a0-\$a3$, $\$v0-\$v1$, $\$gp$, $\$fp$, $\$sp$, $\$ra$, $\$at$	Accesso veloce ai dati. Nel MIPS gli operandi devono essere contenuti nei registri per potere eseguire delle operazioni. Il registro $\$zero$ contiene sempre il valore 0, e il registro $\$at$ viene riservato all'assemblatore per la gestione di costanti molto lunghe.
2 ³⁰ parole di memoria	Memoria[0], Memoria[4], Memoria[4294967292]	Alla memoria si accede solamente attraverso le istruzioni di trasferimento dati. Il MIPS utilizza l'indirizzamento al byte, perciò due parole consecutive hanno indirizzi in memoria a una distanza di 4. La memoria consente di memorizzare strutture dati, vettori, o il contenuto dei registri.

- $\$s0-\$s7$: registri per le variabili dei programmi
- $\$t0-\$t9$: registri temporanei
- $\$zero$: contiene sempre il valore 0

3



Riepilogo istruzioni (2)

Linguaggio assembler MIPS

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	add $\$s1, \$s2, \$s3$	$\$s1 = \$s2 + \$s3$	Operandi in tre registri
	Sottrazione	sub $\$s1, \$s2, \$s3$	$\$s1 = \$s2 - \$s3$	Operandi in tre registri
	Somma immediata	addi $\$s1, \$s2, 20$	$\$s1 = \$s2 + 20$	Utilizzata per sommare delle costanti
Trasferimento dati	Lettura parola	lw $\$s1, 20(\$s2)$	$\$s1 = \text{Memoria}[\$s2 + 20]$	Trasferimento di una parola da memoria a registro
	Memorizzazione parola	sw $\$s1, 20(\$s2)$	$\text{Memoria}[\$s2 + 20] = \$s1$	Trasferimento di una parola da registro a memoria
Logiche	And	and $\$s1, \$s2, \$s3$	$\$s1 = \$s2 \& \$s3$	Operandi in tre registri; AND bit a bit
	Or	or $\$s1, \$s2, \$s3$	$\$s1 = \$s2 \mid \$s3$	Operandi in tre registri; OR bit a bit
	Nor	nor $\$s1, \$s2, \$s3$	$\$s1 = \sim(\$s2 \mid \$s3)$	Operandi in tre registri; NOR bit a bit

4



Riepilogo istruzioni (3)

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Logiche (<i>segue</i>)	And immediato	andi \$s1,\$s2,20	$\$s1 = \$s2 \& 20$	AND bit a bit tra un operando in registro e una costante
	Or immediato	ori \$s1,\$s2,20	$\$s1 = \$s2 20$	OR bit a bit tra un operando in registro e una costante
	Scorrimento logico a sinistra	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	Spostamento a sinistra del numero di bit specificato dalla costante
	Scorrimento logico a destra	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$	Spostamento a destra del numero di bit specificato dalla costante
Salti condizionati	Salta se uguale	beq \$s1,\$s2,25	Se $(\$s1 == \$s2)$ vai a PC+4+100	Test di uguaglianza; salto relativo al PC
	Salta se non è uguale	bne \$s1,\$s2,25	Se $(\$s1 \neq \$s2)$ vai a PC+4+100	Test di disuguaglianza; salto relativo al PC
	Poni uguale a 1 se minore	slt \$s1,\$s2,\$s3	Se $(\$s2 < \$s3)$ $\$s1 = 1$; altrimenti $\$s1 = 0$	Comparazione di minoranza; utilizzata con bne e beq
Salti incondizionati	Salto incondizionato	j 2500	Vai a 10000	Salto all'indirizzo della costante

5



CPUlator (1)

CPUlator è un **simulatore** che permette di eseguire programmi assembly direttamente dal browser.

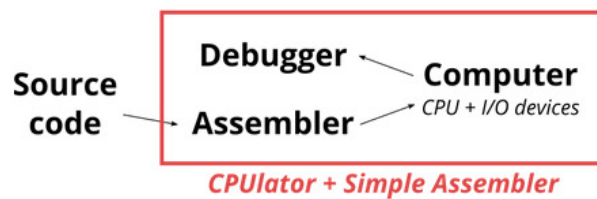
Integra tre componenti fondamentali:

- o **Assembler**: converte il codice sorgente in istruzioni macchina
- o **Computer simulato**: esegue le istruzioni
- o **Debugger**: permette di osservare e analizzare il comportamento del programma

6

CPULator (2)

In un tipico flusso di sviluppo assembly servono:
codice sorgente, assembler, computer che esegue il programma e debugger.
 CPULator li include **tutti in un'unica piattaforma online.**



7

CPULator (3)

Choose a system to simulate

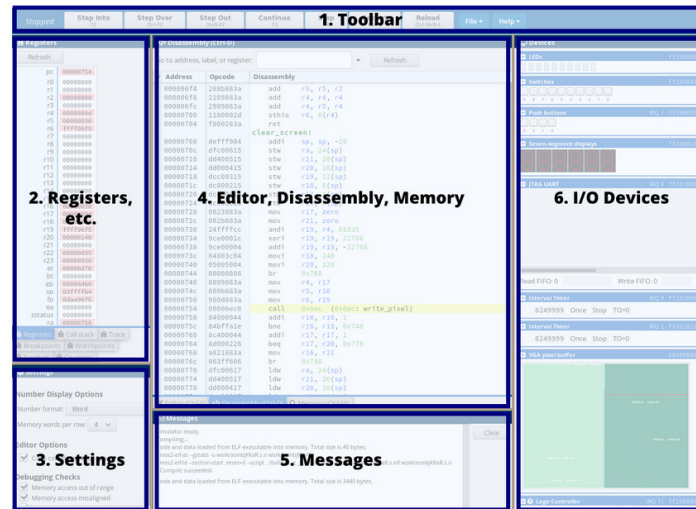
Architecture	System
Any	MIPS generic
Nios II	MIPS SPIM
ARMv7	MIPS (no delay slots) generic
MIPS32r5	MIPS (no delay slots) SPIM
MIPS32r6	Nios II generic
RISC-V RV32	Nios II DE1-SoC
	Nios II DE1-SoC (v16.1)

<https://cpulator.01xz.net/?sys=mipsr5-spim>

- o Architettura **MIPS32r5** per la simulazione del processore
- o Sistema **MIPS SPIM** come ambiente per eseguire ed esaminare i programmi

8

CPUlator (4)



9

Esempio (1)

- Frammento di programma in un linguaggio ad alto livello

```
f = (g + h) - (i + j)
```

- Assembler MIPS

```
add $t0,$s1,$s2 # il registro temporaneo
                  # $t0 contiene il risultato g + h
add $t1,$s3,$s4 # il registro temporaneo
                  # $t1 contiene il risultato i + j
sub $s0,$t0,$t1  # f assume il valore $t0 - $t1,
                  # cioè (g + h) - (i + j)
```

10



Esempio (2)

Indirizzo
di memoria

$g = h + A[8]$

```
lw $t0, 32($s3)    # $t0 prende il
                   # valore di A[8]
                   # 32 = 8x4
add $s1, $s2, $t0  # g = h + A[8]
```

11



Esempio (3)

```
f in $s0          if (i==j) {f=g+h}
g in $s1          else {f=g-h}
h in $s2
i in $s3
j in $s4
```

codice modificato:

```
if (i!=j) {goto Else}
f=g+h
goto Esci
Else: f=g-h
Esci:
```

```
bne $s3, $s4, Else    # se i ≠ j vai a Else
add $s0, $s1, $s2     # f=g+h
j Esci                # vai Esci
Else: sub $s0, $s1, $s2 # f=g-h
Esci:
```

12



Esempio (4)

```
h in $s2
i in $s3
j in $s4
```

```
for (i:=0; i<j; i++) {h=h-1}
codice modificato:
```

```

i=0
Loop: if (i>=j){goto Esci}
      h=h-1
      i=i+1
      goto Loop
Esci:
```

```

add $s3, $zero, $zero # i=0
Loop: slt $t0, $s3, $s4 # se i>=j $t0=0
      beq $t0, $zero, Esci # se i>=j vai a Esci
      addi $s2, $s2, -1 # h=h-1
      addi $s3, $s3, 1 # i=i+1
      j Loop # vai a Loop
Esci:
```

13



Esercizio (1)

Esercizio 6.1. Si assuma che il registro \$s4 contenga l'indirizzo base del vettore A e che il registro \$s0 contenga il valore della variabile i.

1. Si scriva il frammento di codice MIPS (opportunamente commentato) corrispondente al seguente codice Go: `for i=1; i ≤ 3; i++ {A[i]= A[0]}`
2. Quale è il valore esadecimale contenuto nel registro \$s0 una volta eseguito il codice MIPS?

14



Esercizio (2)

Esercizio 6.3. Si assuma che il registro \$s0 contenga l'indirizzo base del vettore A e che i registri \$s1 e \$s2 contengano il valore delle variabili i e j.

1. Si scriva il frammento di codice MIPS (opportunamente commentato) corrispondente all'istruzione: `A[i] +=1`
2. Si scriva il frammento di codice MIPS (opportunamente commentato) corrispondente all'istruzione: `if i<3 && j==0 {i = 2}`

15



Esercizio (3)

Esercizio 6.4. Si assuma che il registro \$t0 contenga l'indirizzo base del vettore A, che il registro \$s1 contenga il valore della variabile i e che il registro \$s2 contenga il valore della variabile j.

11. Si scriva il frammento di codice MIPS (opportunamente commentato) corrispondente al frammento di codice:

```
i=j
if i<2 {A[5] = 3}
else {A[4] = 0}
j = A[6]
```

16



Esercizio (4)

Esercizio 6.5. Esprimere in un linguaggio ad alto livello il seguente frammento di codice. Si assuma che il registro \$s1 contenga la variabile x, il registro \$s2 contenga la variabile y, il registro \$s3 contenga la variabile z, il registro \$s4 contenga la variabile w e il registro \$s5 contenga la variabile v.

```
add $t2, $s1, $s2
sub $t3, $s3, $s4
sub $t4, $t2, $t3
add $t1, $t4, $s5
```

17



Esercizio (5)

Esercizio 6.7. Si assuma che il registro \$s1 contenga l'indirizzo del vettore A di 4 interi, dove A[0]=0, A[1]=1, A[2]=2, A[3]=3. Specificare il contenuto del registro \$s2 al termine delle seguenti istruzioni:

```
lw $s2, 0($s1)
lw $s3, 12($s1)
beq $s2, $s3, EXIT
add $s2, $s3, $s2
EXIT: addi $s2, $s2, 4
```

18